📁 Dataset Structure

Your analysis will use 3 interconnected tables with real e-commerce patterns:

1. Orders Table (15,247 rows)

order_id	customer_id	order_date	order_value	payment_method	status	city
ORD_10001	CUST_2341	2024-01-05	1299	UPI	Delivered	Mumbai
ORD_10002	CUST_5672	2024-01-05	3499	Card	Delivered	Bangalore
ORD_10003	CUST_8923	2024-01-06	599	COD	Returned	Delhi
ORD_10004	CUST_2341	2024-01-08	2199	UPI	Delivered	Mumbai
...	...	...	...	...	...	...

2. Order Items Table (42,891 rows)

order_id	product_id	quantity	price	discount
ORD_10001	PROD_101	1	1299	0%
ORD_10002	PROD_103	1	2499	10%
ORD_10002	PROD_302	2	499	5%
ORD_10003	PROD_567	1	599	0%
...	...	...	...	...

3. Products Table (342 rows)

product_id	product_name	category	sub_category	brand	cost_price
PROD_101	Wireless Headphones	Electronics	Audio	SoundMax	1800

product_id	product_name	category	sub_category	brand	cost_price
	Pro				
PROD_102	Bluetooth Speaker Mini	Electronics	Audio	SoundMax	1200
PROD_103	Smartphone X12	Electronics	Mobiles	TechBrand	15000
PROD_201	Running Shoes Sport	Fashion	Footwear	ActiveFit	900
...	...	...	...	...	...

🎯 Analysis Challenges

Challenge 1: Revenue Analysis

- **Calculate monthly revenue trends** - Is revenue growing or declining?
- **Identify top revenue-generating months** - When do we make the most money?
- **Analyze revenue by payment method** - Which payment methods drive the highest sales?
- **Calculate average order value (AOV)** by city and month
- **Find revenue contribution by category** - Which product categories are most profitable?

Challenge 2: Customer Segmentation (RFM)

- **Calculate Recency** - Days since last purchase for each customer
- **Calculate Frequency** - Number of orders per customer
- **Calculate Monetary** - Total spend per customer
- **Create customer segments:**
  - VIP (High frequency + High monetary)

  - At Risk (High recency + Previously high value)
  - New Customers (Low frequency + Recent purchase)
  - Lost (High recency + Low recent value)
- **Recommend actions** for each segment

Challenge 3: Product Performance

- **Top 10 products by revenue** - What should we stock more of?
- **Top 10 products by units sold** - What's most popular?
- **Calculate profit margins** (price - cost_price) by product
- **Identify underperforming products** - Low sales despite high inventory
- **Cross-sell analysis** - Which products are frequently bought together?
- **Discount effectiveness** - Do discounted products drive more revenue?

Challenge 4: Geographic Insights

- **Revenue by city** - Which cities are our top markets?
- **AOV by city** - Where do customers spend more per order?
- **Return rate by city** - Which cities have the highest returns?
- **Payment method preference by city** - COD vs. Digital payments
- **Growth opportunities** - Which cities have potential for expansion?

Challenge 5: Cohort Analysis

- **Group customers by first purchase month** (cohort)
- **Track retention rate** - What % of customers return each month?

- **Calculate cohort lifetime value** - How much does each cohort spend over time?
- **Identify patterns** - When do customers typically make their second purchase?
- **Churn analysis** - At what point do customers stop buying?

💻 Code Templates to Get Started

SQL Template: Monthly Revenue Trend

```sql
-- Calculate monthly revenue with growth rate
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') as month,
        SUM(order_value) as revenue,
        COUNT(DISTINCT order_id) as total_orders,
        AVG(order_value) as avg_order_value
    FROM orders
    WHERE status = 'Delivered'
    GROUP BY month
    ORDER BY month
)
SELECT
    month,
    revenue,
    total_orders,
    avg_order_value,
    -- Calculate month-over-month growth
    LAG(revenue) OVER (ORDER BY month) as prev_month_revenue,
    ROUND(((revenue - LAG(revenue) OVER (ORDER BY month)) /
           LAG(revenue) OVER (ORDER BY month)) * 100, 2) as growth_percent
FROM monthly_sales;
```

Python Template: Customer Segmentation (RFM)

```python
import pandas as pd
import numpy as np

# Load data
orders = pd.read_csv('orders.csv')
orders['order_date'] = pd.to_datetime(orders['order_date'])

# Calculate RFM metrics
current_date = orders['order_date'].max()

rfm = orders[orders['status'] == 'Delivered'].groupby('customer_id').agg({
    'order_date': lambda x: (current_date - x.max()).days,  # Recency
    'order_id': 'count',                                     # Frequency
    'order_value': 'sum'                                     # Monetary
})

rfm.columns = ['recency', 'frequency', 'monetary']

# Create segments
```

```
rfm['segment'] = 'Regular'
rfm.loc[(rfm['frequency'] >= 5) & (rfm['monetary'] >= 10000), 'segment'] = 'VIP'
rfm.loc[(rfm['recency'] > 90) & (rfm['monetary'] >= 5000), 'segment'] = 'At Risk'
rfm.loc[(rfm['frequency'] == 1) & (rfm['recency'] <= 30), 'segment'] = 'New'
rfm.loc[(rfm['recency'] > 180), 'segment'] = 'Lost'

# Analyze segments
segment_analysis = rfm.groupby('segment').agg({
    'recency': 'mean',
    'frequency': 'mean',
    'monetary': ['mean', 'sum', 'count']
}).round(2)

print(segment_analysis)
```

SQL Template: Product Cross-Sell Analysis

```
-- Find products frequently bought together
SELECT
    a.product_id as product_a,
    b.product_id as product_b,
    COUNT(*) as times_bought_together,
    p1.product_name as product_a_name,
    p2.product_name as product_b_name
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id
    AND a.product_id < b.product_id
JOIN products p1 ON a.product_id = p1.product_id
JOIN products p2 ON b.product_id = p2.product_id
GROUP BY a.product_id, b.product_id, p1.product_name, p2.product_name
HAVING times_bought_together >= 10
ORDER BY times_bought_together DESC
LIMIT 20;
```

📝 Deliverables

Create a comprehensive analysis report that includes:

1. **Executive Summary** - Key findings in 5 bullet points
2. **Revenue Analysis** - Trends, patterns, and insights
3. **Customer Segments** - RFM analysis with actionable recommendations
4. **Product Insights** - Top performers, cross-sell opportunities
5. **Geographic Analysis** - City-wise performance and opportunities
6. **Recommendations** - 5-7 data-driven actions to increase revenue by 20%

Bonus: Create visualizations (charts/graphs) to support your findings!

✅ Success Criteria

- ✓ All 5 challenges completed with SQL/Python code
- ✓ Clear insights derived from data analysis
- ✓ Actionable recommendations backed by data
- ✓ Code is well-commented and reproducible
- ✓ Professional presentation of findings

Portfolio Tip: This project demonstrates real Business Analyst skills. Add it to your resume and GitHub to impress recruiters!